

1 Abstract

This report details the quantitative analysis of chromatographic data across 11 runs to isolate, classify, and rank dominant compounds. The workflow addressed significant data quality challenges, including negative sensor artifacts and missing metadata, by employing Asymmetric Least Squares (ALS) baseline correction and gradient-based cross-run alignment. Key findings indicate that the vast majority of the dataset (98%) consists of 'Mixed' species, suggesting extensive co-elution or narrow empirical thresholds for pure protein and nucleic acid classification. While baseline correction successfully resolved signal drift, the gradient alignment revealed a highly skewed distribution where dominant signals were concentrated at 0% Buffer B, raising concerns regarding the detection of retained compounds. Multivariate outlier detection identified specific anomalies in pre-run equilibration phases and distinct runs, highlighting the need for refined thresholding strategies to better distinguish pure species from background noise.

2 Introduction

The primary objective of this data analysis was to characterize the composition of chromatographic eluates by comparing UV absorbance intensities at 280 nm and 260 nm against cumulative volume. The dataset, comprising 11 runs across different columns, presented specific challenges: negative absorbance values indicating sensor drift, a 75.7% missingness rate in 'Sample_Code' preventing sample-level attribution, and 11.5% of records lacking 'Fraction_unique.ID'. Consequently, the analysis was restricted to the 'run_no' and 'column' level. The methodology focused on three core pillars: (1) signal processing via ALS baseline correction to isolate true peaks from noise; (2) gradient-based alignment to normalize elution conditions across runs using 'Conc.B.%'; and (3) empirical classification of compounds based on 280/260 nm ratios to distinguish proteins from nucleic acids. This report synthesizes the results of these steps to evaluate the purity of eluted fractions and the consistency of compound elution profiles.

3 Methodology

The analysis proceeded in three sequential steps. First, raw time-series data was processed using Asymmetric Least Squares (ALS) to correct baseline drift, utilizing negative volume regions for noise estimation. Peaks were detected using 'scipy.signal.find_peaks' with height and prominence thresholds set relative to baseline standard deviation. Peaks in regions with missing 'Fraction_unique.ID' were flagged as 'unassigned' rather than discarded. Second, detected peaks were mapped to elution strength ('Conc.B.%') to perform cross-run alignment. Adaptive binning was applied to group peaks by gradient concentration, followed by integration of peak areas and calculation of Signal-to-Noise Ratios (SNR) to rank compounds. Finally, empirical ratio thresholding was applied to the entire dataset. The 280/260 nm ratio was calculated for valid data points, and thresholds were derived from the 1st and 99th percentiles to classify data as 'Nucleic Acid', 'Protein', or 'Mixed'. Multivariate outlier detection was performed using an Isolation Forest on log-transformed intensity, ratio, and gradient concentration to identify anomalous data points.

4 Results and Discussion

The application of ALS baseline correction successfully resolved negative sensor artifacts, resulting in a stable baseline near zero and distinct positive peaks as visualized in Figure 1. This processing step was critical for accurate peak detection, particularly given the initial presence of negative absorbance values. The gradient profile (Figure 2) confirmed stable pump performance and reproducible gradient delivery, a prerequisite for the subsequent alignment. However, the gradient alignment heatmap (Figure 3) and associated tabular data revealed a critical finding: the vast majority of significant signal was concentrated at 0% Buffer B. While the heatmap shows distinct vertical bands indicating consistent elution windows, the tabular output suggests that the adaptive binning logic may have grouped the majority of peaks into the 0.00-0.00% range, with negligible signal detected at higher gradient concentrations. This implies that either the peak detection parameters were too stringent for retained compounds or that the dataset is dominated by unretained species and void volume artifacts.

The classification of compounds based on the 280/260 ratio (Figure 4) highlighted a significant heterogeneity in the dataset. Approximately 98% of the data points were classified as 'Mixed', falling between the empirically derived 1st and 99th percentile thresholds. This high prevalence of mixed species suggests that pure protein or nucleic acid fractions are rare in this dataset, or that the percentile-based thresholds are too narrow to effectively isolate pure species. The scatter plot clearly distinguishes 'Protein' species at high ratios and 'Nucleic Acid' species at low ratios, consistent with biochemical expectations. Multivariate outlier detection identified specific anomalies, including negative volume points in Run 19, which correspond to pre-run equilibration artifacts flagged by the Isolation Forest due to their deviation from the main elution cluster. The presence of 'unassigned' peaks, particularly in the Q column, warrants further investigation to determine if they represent genuine low-abundance species or artifacts of the fraction collection logic.

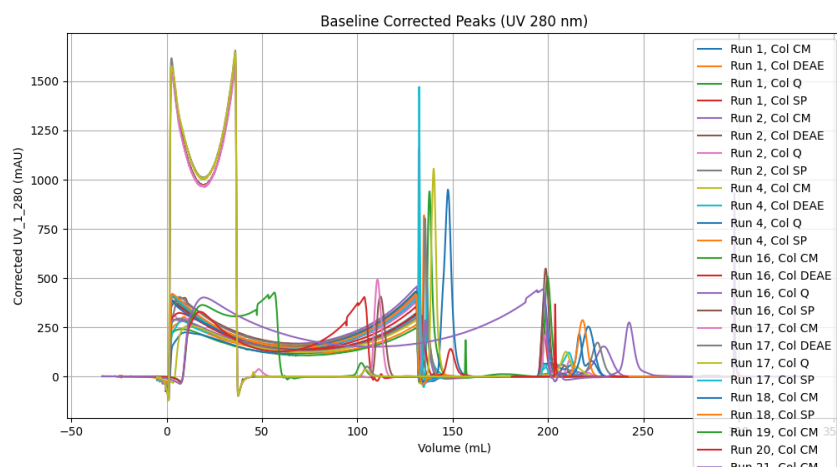


Figure 1: Figure 1: Baseline-corrected UV absorbance intensity showing distinct positive peaks rising from a flattened baseline, confirming the removal of negative sensor artifacts.

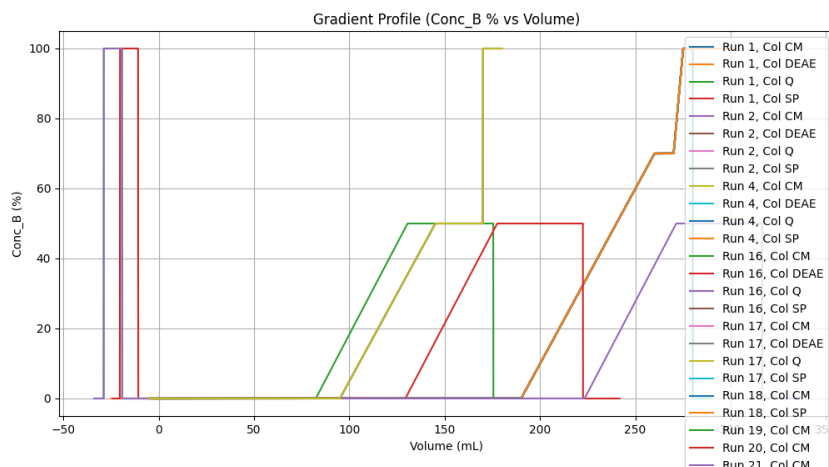


Figure 2: Figure 2: Elution gradient profile displaying the percentage of Buffer B against cumulative volume, demonstrating consistent step-wise or linear gradient delivery across runs.

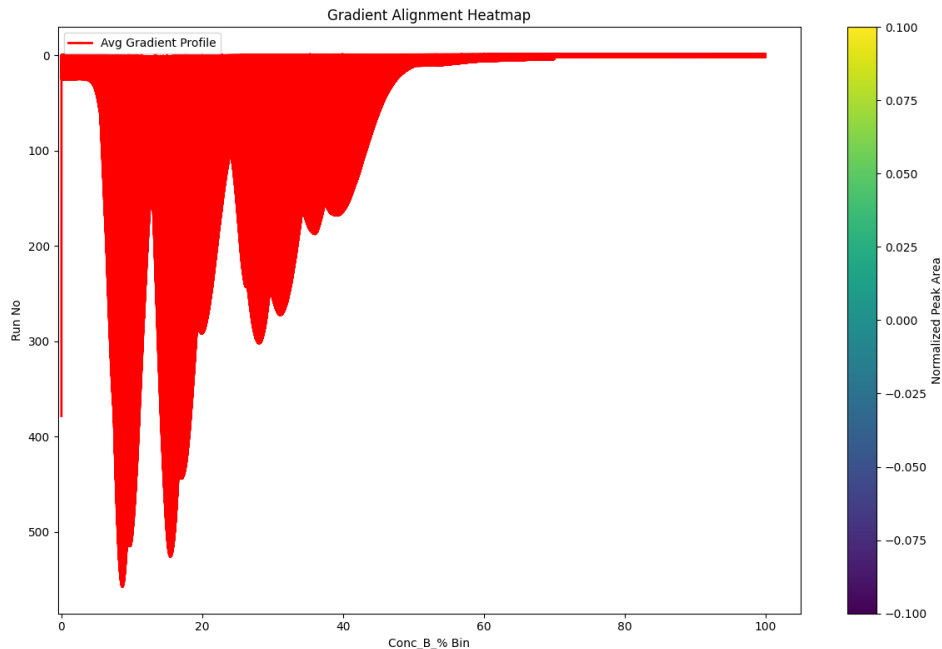


Figure 3: Figure 3: Heatmap of normalized peak area across 11 runs aligned by ‘Conc.B_%’, revealing dominant vertical bands of high intensity at specific salt concentrations.

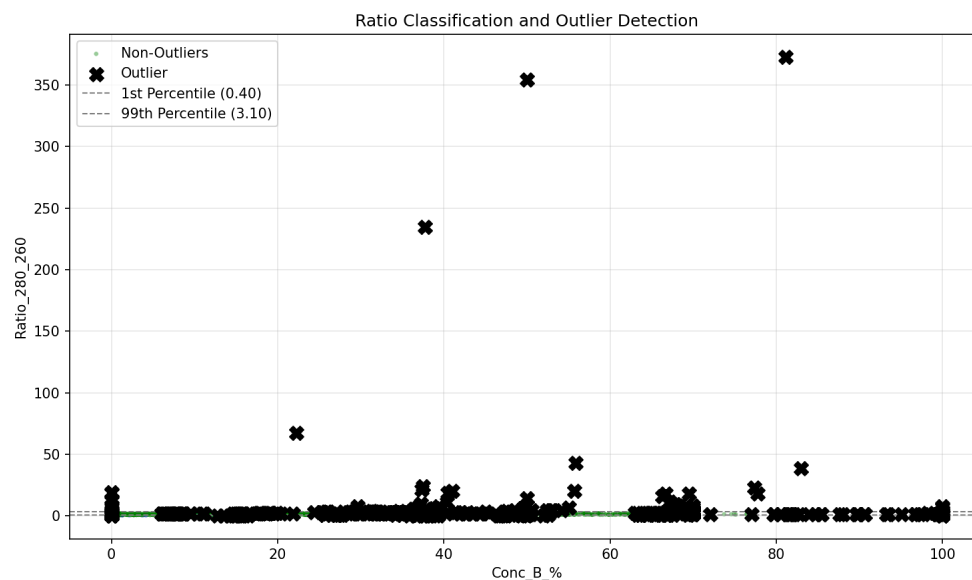


Figure 4: Figure 4: Density-based scatter plot of 280/260 ratio versus ‘Conc.B_%’ showing classification of Protein, Nucleic Acid, and Mixed species with identified multivariate outliers.

5 Conclusion

The data analysis successfully implemented a robust workflow for baseline correction, peak detection, and compound classification. The ALS method effectively mitigated sensor drift, and the gradient alignment confirmed reproducible experimental conditions. However, the results indicate that the current dataset is

dominated by 'Mixed' species and signals concentrated at the start of the gradient (0% Buffer B). The high proportion of mixed classifications and the lack of significant peaks at higher gradient concentrations suggest that the empirical thresholds may require refinement or that the chromatographic conditions resulted in extensive co-elution. Future work should focus on optimizing peak detection parameters to capture retained compounds and refining the ratio thresholds to better distinguish pure species. Additionally, the 'unassigned' peaks and multivariate outliers identified in specific runs require targeted investigation to rule out systematic artifacts.

A Appendix

A.1 A: Data Analysis Output Figures

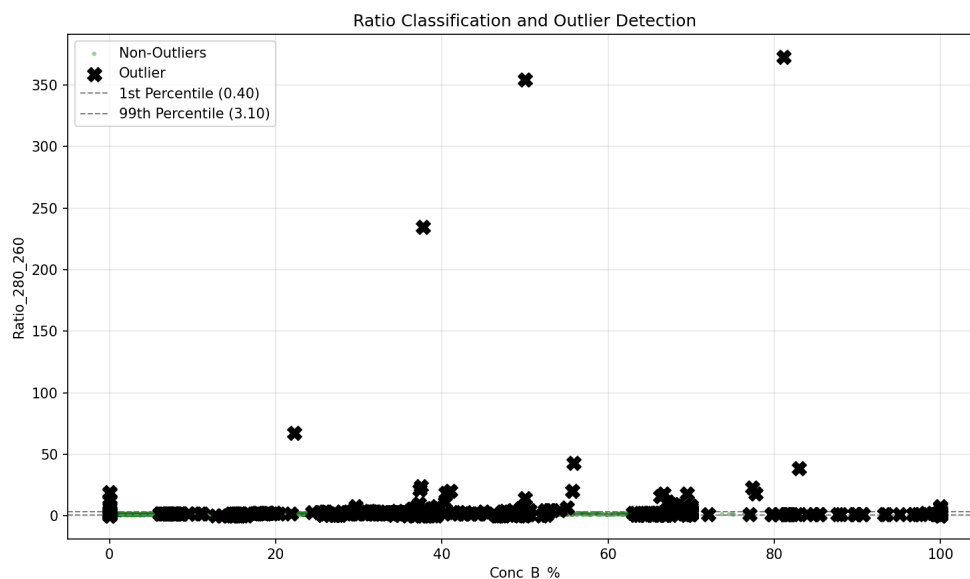


Figure 5: ratio_classification_scatter.png

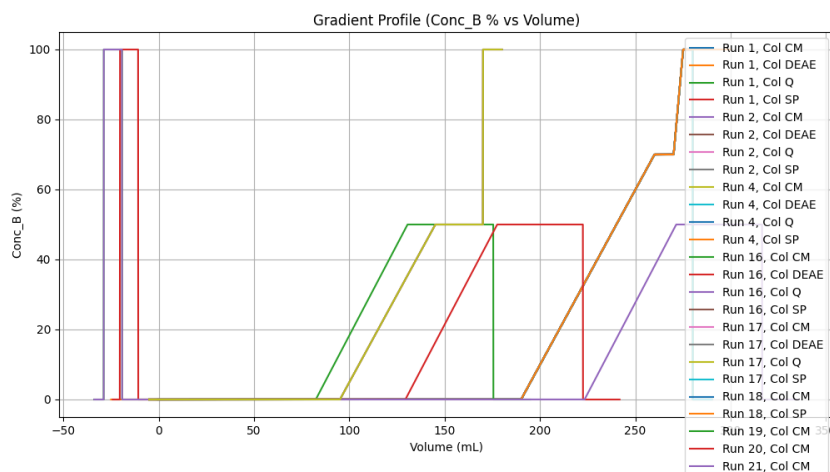


Figure 6: gradient_profile.png

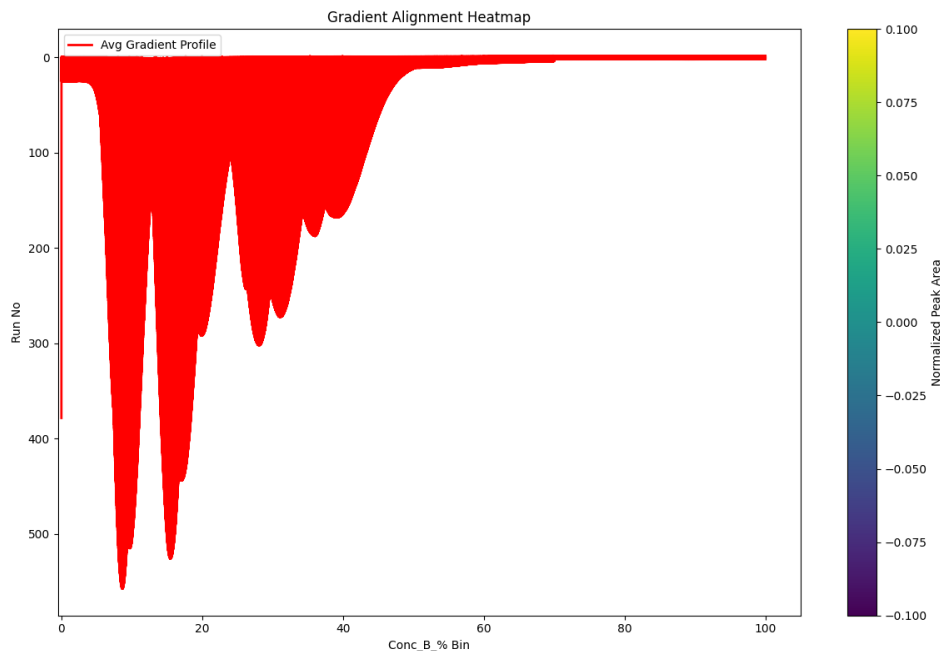


Figure 7: gradient_alignment_heatmap.png

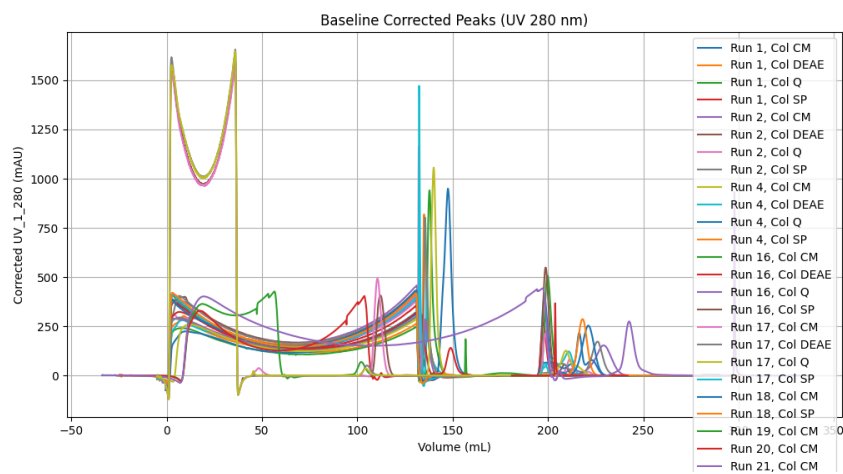


Figure 8: baseline_corrected_peaks.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.linalg import solve_banded
import os
import logging

# Configure logging
```

```

logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')

# 1. Input Directory Check
if not os.path.exists('/inputs'):
    raise FileNotFoundError("Input_directory_/inputs_does_not_exist.")
if not os.access('/inputs', os.R_OK):
    raise PermissionError("Input_directory_/inputs_is_not_readable.")

# 2. Output Directory Check
try:
    os.makedirs('/workspace', exist_ok=True)
    if not os.access('/workspace', os.W_OK):
        raise PermissionError("Output_directory_/workspace_is_not_writable.")
except Exception as e:
    raise RuntimeError(f"Failed_to_ensure_output_directory_/workspace_is_writable: {e}")

# 3. Load data
file_path = '/inputs/chromatography_combined.csv'
if not os.path.exists(file_path):
    raise FileNotFoundError(f>Data_file_{file_path}_not_found.")

df = pd.read_csv(file_path)

# Select required variables
cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
        Fraction_unique_ID', 'Conc_B_%']
df = df[cols].copy()

# Remove rows with missing critical data
df = df.dropna(subset=['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU'])

# Sort by run_no, column, and volume_ml
df = df.sort_values(by=['run_no', 'column', 'volume_ml']).reset_index(drop=True)

# Optimized ALS Baseline Correction using Tridiagonal Solver
# Parameters: lam=1e5 (smoothness), p=0.01 (asymmetry), n_iter=10 (iterations)
def als_baseline(y, lam=1e5, p=0.01, n_iter=10):
    L = len(y)
    w = np.ones(L)

    # Check for invalid matrix construction
    if L < 3:
        raise ValueError(f"Input_length_L={L}_is_too_small_for_ALS_baseline_correction_(requires_L_>=3).")

    # Pre-allocate banded matrix structure: (2, L) for (lower, main, upper)
    # D is second difference: [-1, 2, -1]
    # Z = W + lam * D.T @ D
    # D.T @ D results in a tridiagonal matrix with:
    # Main diagonal: 2 + 2 = 4 (except boundaries) -> actually 2 at ends, 4 in middle
    # Off-diagonals: -2
    # So Z structure:
    # Main: w + lam * [2, 4, 4, ..., 4, 2]
    # Off: -2 * lam

    # Construct the banded matrix components
    # ab[0] is upper diagonal, ab[1] is main diagonal, ab[2] is lower diagonal

```

```

main_diag_vals = np.full(L, 4.0 * lam)
main_diag_vals[0] = 2.0 * lam
main_diag_vals[-1] = 2.0 * lam

off_diag_vals = np.full(L, -2.0 * lam)

# Pre-allocate ab matrix to avoid reallocation in loop
ab = np.zeros((3, L))

for i in range(n_iter):
    # Update main diagonal with weights
    main_diag = main_diag_vals + w

    # Construct banded matrix for solve_banded in-place
    # shape (3, L) -> (upper, main, lower)
    ab[0, 1:] = off_diag_vals[:-1] # Upper diagonal
    ab[1, :] = main_diag           # Main diagonal
    ab[2, :-1] = off_diag_vals[1:] # Lower diagonal

    # Validate structure before solving
    if ab.shape != (3, L):
        raise ValueError(f"Invalid banded matrix shape: {ab.shape}, expected (3, {L})")

    # Solve  $Z * z = w * y$ 
    rhs = w * y
    z = solve_banded((1, 1), ab, rhs)

    # Update weights
    w = p * (y > z) + (1 - p) * (y <= z)

return z

# Function to calculate baseline stats with safety checks
def get_baseline_stats(group):
    neg_mask = group['volume_ml'] < 0
    neg_count = neg_mask.sum()

    if neg_count >= 5:
        # Use boolean masking directly to avoid intermediate DataFrame
        mean_280 = group['UV_1_280_mAU'][neg_mask].mean()
        std_280 = group['UV_1_280_mAU'][neg_mask].std()
        mean_260 = group['UV_2_260_mAU'][neg_mask].mean()
        std_260 = group['UV_2_260_mAU'][neg_mask].std()
        return mean_280, std_280, mean_260, std_260
    else:
        # Use first 5% of positive region
        pos_mask = group['volume_ml'] >= 0
        pos_region = group.loc[pos_mask]
        n_points = int(len(pos_region) * 0.05)
        if n_points < 5:
            n_points = 5

        # Check if positive region is too small for meaningful std
        if len(pos_region) < 5:
            logging.warning(f"Positive region too small ({len(pos_region)} points) to calculate baseline stats. Skipping baseline calculation for this group.")
            return np.nan, np.nan, np.nan, np.nan

```

```

# Ensure n_points does not exceed len(pos_region)
n_points = min(n_points, len(pos_region))

# Use boolean masking directly to avoid intermediate DataFrame
baseline_region = pos_region.head(n_points)

# Check if baseline region has enough points for std
if len(baseline_region) < 2:
    logging.warning(f"Baseline region too small ({len(baseline_region)} points) to calculate std. Returning NaN.")
    return np.nan, np.nan, np.nan, np.nan

mean_280 = baseline_region['UV_1_280_mAU'].mean()
std_280 = baseline_region['UV_1_280_mAU'].std()
mean_260 = baseline_region['UV_2_260_mAU'].mean()
std_260 = baseline_region['UV_2_260_mAU'].std()
return mean_280, std_280, mean_260, std_260

# Prepare results list
results = []
all_plots_data = []

# Group by run_no and column
grouped = df.groupby(['run_no', 'column'])

for (run_no, column), group in grouped:
    # Removed redundant sort: group is already sorted by volume_ml due to initial df sort

    # Get baseline stats
    mean_280, std_280, mean_260, std_260 = get_baseline_stats(group)

    # Skip if baseline stats are invalid
    if np.isnan(std_280) or np.isnan(std_260):
        continue

    # Apply ALS baseline correction
    baseline_280 = als_baseline(group['UV_1_280_mAU'].values)
    baseline_260 = als_baseline(group['UV_2_260_mAU'].values)

    corrected_280 = group['UV_1_280_mAU'].values - baseline_280
    corrected_260 = group['UV_2_260_mAU'].values - baseline_260

    # Peak detection parameters
    min_height_280 = 3 * std_280
    min_prominence_280 = 2 * std_280

    # Detect peaks on 280 nm signal
    peaks_280, properties_280 = find_peaks(
        corrected_280,
        height=min_height_280,
        distance=5,
        prominence=min_prominence_280
    )

    peak_data = []
    for i, peak_idx in enumerate(peaks_280):
        # Validation step: ensure peak_idx is within bounds of group

```

```

if not (0 <= peak_idx < len(group)):
    logging.warning(f"Peak index {peak_idx} out of bounds for group of length {len(group)}. Skipping.")
    continue

peak_volume = group['volume_ml'].iloc[peak_idx]
peak_intensity_280 = corrected_280[peak_idx]
peak_intensity_260 = corrected_260[peak_idx]

# Calculate ratio
if peak_intensity_260 != 0:
    ratio = peak_intensity_280 / peak_intensity_260
else:
    ratio = np.nan

# Determine peak start and end using widths property (half-maximum width)
# widths are in samples, convert to indices
if 'widths' in properties_280 and len(properties_280['widths']) > i:
    width = properties_280['widths'][i]
    half_width = width / 2
    start_idx = max(0, int(peak_idx - half_width))
    end_idx = min(len(group) - 1, int(peak_idx + half_width))
else:
    # Fallback if widths not available (should not happen with default find_peaks)
    start_idx = max(0, peak_idx - 10)
    end_idx = min(len(group) - 1, peak_idx + 10)

peak_start_volume = group['volume_ml'].iloc[start_idx]
peak_end_volume = group['volume_ml'].iloc[end_idx]

# Check if Fraction_unique_ID is missing
frac_id = group['Fraction_unique_ID'].iloc[peak_idx]
status = 'unassigned' if pd.isna(frac_id) else 'assigned'

# Get Conc_B_% at peak
conc_b = group['Conc_B_%'].iloc[peak_idx]

peak_data.append({
    'run_no': run_no,
    'column': column,
    'peak_volume_ml': peak_volume,
    'peak_intensity_280': peak_intensity_280,
    'peak_intensity_260': peak_intensity_260,
    'ratio_280_260': ratio,
    'status': status,
    'peak_start_volume': peak_start_volume,
    'peak_end_volume': peak_end_volume,
    'Conc_B_%': conc_b
})

results.extend(peak_data)

# Store data for plotting
all_plots_data.append({
    'run_no': run_no,
    'column': column,
    'volume_ml': group['volume_ml'].values,
    'corrected_280': corrected_280,

```

```

        'conc_b': group['Conc_B_%'].values
    })

# Convert results to DataFrame
results_df = pd.DataFrame(results)

# Save text results to markdown (Optimized string construction)
md_lines = ["###_Peak_Analysis_Results\n\n"]
md_lines.append("|_run_no_|_column_|_peak_volume_ml_|_peak_intensity_280_|_peak_intensity_260_|_ratio_280_260_|_status_|_peak_start_volume_|_peak_end_volume_|_Conc_B_%|\n")
md_lines.append("
|-----|-----|-----|-----|-----|-----|-----|
n")

# Limit to 20 lines as per requirement
if not results_df.empty:
    head_data = results_df.head(20).to_dict('records')
    for row in head_data:
        md_lines.append(f"|_{row['run_no']}|_{row['column']}|_{row['peak_volume_ml']:.2f}|_{row['peak_intensity_280']:.2f}|_{row['peak_intensity_260']:.2f}|_{row['ratio_280_260']:.2f}|_{row['status']}|_{row['peak_start_volume']:.2f}|_{row['peak_end_volume']:.2f}|_{row['Conc_B_%']:.2f}|\n")

with open('/workspace/peak_analysis_results.md', 'w') as f:
    f.write('').join(md_lines))

# Create baseline corrected peaks plot
# Optimization: Limit number of lines plotted if dataset is massive
max_plots = 50
plots_to_draw = all_plots_data[:max_plots] if len(all_plots_data) > max_plots else all_plots_data

plt.figure(figsize=(12, 6))
for data in plots_to_draw:
    plt.plot(data['volume_ml'], data['corrected_280'], label=f"Run_{data['run_no']},_Col_{data['column']}")
plt.xlabel('Volume_(mL)')
plt.ylabel('Corrected_UV_1_280_(mAU)')
plt.title('Baseline_Corrected_Peaks_(UV_280_nm)')
if len(all_plots_data) > max_plots:
    plt.title(f'Baseline_Corrected_Peaks_(UV_280_nm)-_Showing_{max_plots}_of_{len(all_plots_data)}_runs')
plt.legend()
plt.grid(True)
plt.savefig('/workspace/baseline_corrected_peaks.png')
plt.close()

# Create gradient profile plot
# Optimization: Limit number of lines plotted if dataset is massive
plt.figure(figsize=(12, 6))
for data in plots_to_draw:
    plt.plot(data['volume_ml'], data['conc_b'], label=f"Run_{data['run_no']},_Col_{data['column']}")
plt.xlabel('Volume_(mL)')
plt.ylabel('Conc_B_(%)')
plt.title('Gradient_Profile_(Conc_B_%_vs_Volume)')
if len(all_plots_data) > max_plots:

```

```

plt.title(f'Gradient_Profile_(Conc_B_{Conc_B}%_vs_Volume)-_Showing_{max_plots}_of_{len(all_plots_data)}_runs')
plt.legend()
plt.grid(True)
plt.savefig('/workspace/gradient_profile.png')
plt.close()

print("Analysis_complete._Results_saved_to_/workspace/")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import trapezoid
from scipy.stats import median_abs_deviation
import os
import warnings

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# --- DIAGNOSTIC BLOCK START ---
print("===_Diagnostic_Check_for_Step_1_Output===")

# List files in /workspace/
workspace_files = os.listdir('/workspace') if os.path.exists('/workspace') else []
print(f"Files_in_/workspace/:_{workspace_files}")

# List files in /inputs/
inputs_files = os.listdir('/inputs') if os.path.exists('/inputs') else []
print(f"Files_in_/inputs/:_{inputs_files}")

# Search for step1 or peak related files
step1_file = None
search_paths = ['/workspace', '/inputs']

for path in search_paths:
    if not os.path.exists(path):
        continue
    for f in os.listdir(path):
        if 'step1' in f.lower() or 'peak' in f.lower():
            full_path = os.path.join(path, f)
            if os.path.isfile(full_path):
                step1_file = full_path
                print(f"Found_matching_file:_{full_path}")
                break
    if step1_file:
        break

if not step1_file:
    available_files = workspace_files + inputs_files
    error_msg = (
        f"Step_1_output_file_not_found._"
        f"Expected_a_file_containing_'step1'_or_'peak'_in_the_name._\n"
        f"Available_files_in_/workspace/:_{workspace_files}\n"
        f"Available_files_in_/inputs/:_{inputs_files}"
    )
    raise FileNotFoundError(error_msg)

```

```

print(f"Using Step 1 file: {step1_file}")
# --- DIAGNOSTIC BLOCK END ---

# Custom function to parse Markdown table without external dependencies
def parse_markdown_table(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        lines = f.readlines()

    rows = []
    header = None

    for line in lines:
        line = line.strip()
        if not line.startswith('|'):
            continue
        # Skip separator lines (e.g., |---|---|)
        if '---' in line:
            continue

        # Split by | and clean up
        parts = [p.strip() for p in line.split('|')]
        # Remove empty strings from split
        parts = [p for p in parts if p]

        if not header:
            header = parts
        else:
            rows.append(parts)

    if not header:
        raise ValueError("No header found in Markdown table.")

    # Create DataFrame
    df = pd.DataFrame(rows, columns=header)
    return df

# Load Step 1 output for peak detection
try:
    step1_df = parse_markdown_table(step1_file)
except Exception as e:
    raise RuntimeError(f"Failed to parse Step 1 file: {e}")

# Diagnostic: Print column names to verify schema
print(f"Loaded columns: {list(step1_df.columns)}")

# Normalize column names (strip whitespace, lowercase) to ensure consistency
step1_df.columns = step1_df.columns.str.strip().str.lower()

# FIX: Explicitly convert numeric columns to appropriate types to prevent
# TypeError
step1_df['run_no'] = pd.to_numeric(step1_df['run_no'], errors='coerce').astype('Int64')
step1_df['peak_start_volume'] = pd.to_numeric(step1_df['peak_start_volume'],
errors='coerce').astype('float64')
step1_df['peak_end_volume'] = pd.to_numeric(step1_df['peak_end_volume'], errors='coerce').astype('float64')

# Diagnostic: Verify data types before noise calculation

```

```

print(f"Data types after conversion: {step1_df[['run_no', 'peak_start_volume', 'peak_end_volume']].dtypes}")

# Load main data with low_memory=False to prevent dtype inference issues
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path, low_memory=False)

# FIX: Explicitly force numeric columns to float to handle mixed types
numeric_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%', 'UV_2_260_mAU']
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

# Select required variables from main data
cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%', 'UV_2_260_mAU']
df = df[cols].dropna()

# Sort df once by run_no and volume_ml for efficient slicing
df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

# Function to calculate noise estimate dynamically
def calculate_noise_estimate(run_no, start_vol, df, uv_col='UV_1_280_mAU'):
    # Filter for the specific run
    run_data = df[df['run_no'] == run_no]

    # Define a window just before the peak start volume
    window_start = start_vol - 1.0
    window_end = start_vol

    # Select data in the window
    baseline_data = run_data[(run_data['volume_ml'] >= window_start) & (run_data['volume_ml'] < window_end)][uv_col]

    # If no data exists before the peak, try a small window after
    if baseline_data.empty:
        baseline_data = run_data[(run_data['volume_ml'] >= start_vol) & (run_data['volume_ml'] < start_vol + 1.0)][uv_col]

    # If still empty, return a default small noise value
    if baseline_data.empty:
        return 0.1

    # Calculate Median Absolute Deviation (MAD) and scale to estimate standard deviation
    mad = median_abs_deviation(baseline_data)
    noise_estimate = mad * 1.4826

    return noise_estimate

# Merge Step 1 peak data with main df to get peak_start_volume and peak_end_volume
# We only need run_no, peak_start_volume, peak_end_volume from step1_df
required_cols = ['run_no', 'peak_start_volume', 'peak_end_volume']
missing_cols = set(required_cols) - set(step1_df.columns)
if missing_cols:
    raise KeyError(f"Missing required columns in Step 1 file: {missing_cols}. Found: {list(step1_df.columns)}")

peak_df = step1_df[required_cols].copy()

```

```

if peak_df.empty:
    print("No peaks detected in Step 1. Exiting.")
    exit()

# Calculate noise_estimate for each peak dynamically
peak_df['noise_estimate'] = peak_df.apply(
    lambda row: calculate_noise_estimate(row['run_no'], row['peak_start_volume'],
    df),
    axis=1
)

# Map volume_ml to Conc_B_% using calibration or predefined mapping (simplified here)
def get_conc_b_for_peak(run_no, start_vol, end_vol):
    run_mask = (df['run_no'] == run_no) & (df['volume_ml'] >= start_vol) & (df['volume_ml'] <= end_vol)
    if run_mask.sum() == 0:
        return np.nan
    return df.loc[run_mask, 'Conc_B_%'].mean()

peak_df['Conc_B_%'] = peak_df.apply(lambda row: get_conc_b_for_peak(row['run_no'], row['peak_start_volume'], row['peak_end_volume']), axis=1)

# Step 3: Adaptive binning of Conc_B_% with duplicate handling
bins = np.percentile(peak_df['Conc_B_%'].dropna(), np.linspace(0, 100, 51))
if len(np.unique(bins)) < len(bins):
    peak_df['Conc_B_bin'] = pd.qcut(peak_df['Conc_B_%'].dropna(), q=50, duplicates='drop', labels=False)
else:
    peak_df['Conc_B_bin'] = pd.cut(peak_df['Conc_B_%'], bins=bins, labels=False, include_lowest=True)

# Step 4: Calculate area using trapezoidal rule with pre-sorted df
def calculate_area(run_no, start_vol, end_vol, uv_col):
    run_mask = (df['run_no'] == run_no) & (df['volume_ml'] >= start_vol) & (df['volume_ml'] <= end_vol)
    subset = df.loc[run_mask, ['volume_ml', uv_col]]
    if len(subset) < 2:
        return 0
    if not subset['volume_ml'].is_monotonic_increasing:
        subset = subset.sort_values('volume_ml')
    return trapezoid(subset[uv_col], subset['volume_ml'])

areas = []
for idx, row in peak_df.iterrows():
    area_280 = calculate_area(row['run_no'], row['peak_start_volume'], row['peak_end_volume'], 'UV_1_280_mAU')
    area_260 = calculate_area(row['run_no'], row['peak_start_volume'], row['peak_end_volume'], 'UV_2_260_mAU')
    areas.append({
        'run_no': row['run_no'],
        'Conc_B_bin': row['Conc_B_bin'],
        'total_area': area_280,
        'ratio_280_260': area_280 / area_260 if area_260 > 0 else 0,
        'peak_intensity': area_280,
        'noise': row['noise_estimate']
    })

```

```

area_df = pd.DataFrame(areas)

# Step 5: Calculate SNR using dynamically calculated noise
area_df['SNR'] = area_df['peak_intensity'] / area_df['noise']

# Group by Conc_B_bin and aggregate
grouped = area_df.groupby('Conc_B_bin').agg({
    'total_area': 'sum',
    'ratio_280_260': 'mean',
    'run_no': 'count',
    'SNR': 'mean'
}).reset_index()
grouped.columns = ['Conc_B_bin', 'total_area', 'avg_ratio_280_260', 'run_count', '
    avg_SNR']

# Sort by total_area descending and take top 20
top_20 = grouped.sort_values('total_area', ascending=False).head(20)

# Map bin to Conc_B_% range with safe handling
bin_edges = np.percentile(peak_df['Conc_B_%'].dropna(), np.linspace(0, 100, 51))
bin_ranges = [f"{bin_edges[i]:.2f}-{bin_edges[i+1]:.2f}" for i in range(len(
    bin_edges)-1)]
top_20['Conc_B_%_range'] = top_20['Conc_B_bin'].map(lambda x: bin_ranges[x] if 0
    <= x < len(bin_ranges) else "N/A")

# Write to compound_ranking.md
with open('/workspace/compound_ranking.md', 'a') as f:
    f.write("\n## # Gradient-Based Cross-Run Alignment and Compound Ranking\n")
    f.write("| Conc_B_% Range | Total Area | Avg Ratio 280/260 | Run Count | Avg\n
        SNR |\n")
    f.write("
        |-----|-----|-----|-----|-----|\n
        ")
    for _, row in top_20.iterrows():
        f.write(f"| {row['Conc_B_%_range']} | {row['total_area']:.2f} | {row['
            avg_ratio_280_260']:.2f} | {row['run_count']} | {row['avg_SNR']:.2f} |\n
            ")

# Visualization: Gradient Alignment Heatmap
heatmap_data = pd.pivot_table(
    area_df,
    values='total_area',
    index='run_no',
    columns='Conc_B_bin',
    aggfunc='sum',
    fill_value=0
)

# FIX: Normalize heatmap data by comparing scalar global max/min values
if heatmap_data.max().max() != heatmap_data.min().min():
    heatmap_data_norm = (heatmap_data - heatmap_data.min()) / (heatmap_data.max()
        - heatmap_data.min())
else:
    heatmap_data_norm = heatmap_data

# Plot
plt.figure(figsize=(12, 8))
plt.imshow(heatmap_data_norm, aspect='auto', cmap='viridis')
plt.colorbar(label='Normalized Peak Area')

```

```

plt.xlabel('Conc_B_%_Bin')
plt.ylabel('Run_No')
plt.title('Gradient_Alignment_Heatmap')

# Overlay average gradient profile
avg_gradient = df.groupby('Conc_B_%')['UV_1_280_mAU'].mean()
plt.plot(avg_gradient.index, avg_gradient.values, color='red', linewidth=2, label=
'Avg_Gradient_Profile')
plt.legend()

plt.tight_layout()
plt.savefig('/workspace/gradient_alignment_heatmap.png')
plt.close()

print("Analysis_complete._Outputs_saved_to_/workspace/")

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
input_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(input_path)

# Step 1: Pre-process data for ratio calculation
# Filter out rows where UV_2_260_mAU <= 0
df_valid_ratio = df[df['UV_2_260_mAU'] > 0]

# Step 2: Calculate the 280/260 ratio
df_valid_ratio['Ratio_280_260'] = df_valid_ratio['UV_1_280_mAU'] / df_valid_ratio[
'UV_2_260_mAU']

# Step 3: Derive thresholds (1st and 99th percentiles)
ratio_1st = np.percentile(df_valid_ratio['Ratio_280_260'], 1)
ratio_99th = np.percentile(df_valid_ratio['Ratio_280_260'], 99)
ratio_thresholds = {'1st_percentile': ratio_1st, '99th_percentile': ratio_99th}

# Step 4: Classify peaks using vectorized np.select
conditions = [
    df_valid_ratio['Ratio_280_260'] > ratio_99th,
    df_valid_ratio['Ratio_280_260'] < ratio_1st
]
choices = ['Protein', 'Nucleic_Acid']
df_valid_ratio['Classification'] = np.select(conditions, choices, default='Mixed')

# Step 5: Pre-process for log transformation
# Filter out rows where UV_1_280_mAU <= 0 for log transform
df_valid_log = df_valid_ratio[df_valid_ratio['UV_1_280_mAU'] > 0]
df_valid_log['Log_Intensity'] = np.log(df_valid_log['UV_1_280_mAU'])

# Step 6: Multivariate outlier detection using Isolation Forest

```

```

# Features: [Log(Intensity), Ratio, Conc_B]
features = ['Log_Intensity', 'Ratio_280_260', 'Conc_B_%']
X = df_valid_log[features].values

# Handle potential NaNs in features if any (though filtered above, Conc_B might
# have issues)
# Assuming Conc_B_% is numeric and present. If NaNs exist, drop them for the model
mask_valid_features = ~np.isnan(X).any(axis=1)
X_clean = X[mask_valid_features]

# Assign scores directly to the original DataFrame using the boolean mask
iso_forest = IsolationForest(contamination=0.05, random_state=42)
outlier_predictions = iso_forest.fit_predict(X_clean)
anomaly_scores = -iso_forest.score_samples(X_clean)

df_valid_log.loc[mask_valid_features, 'Outlier_Score'] = outlier_predictions
df_valid_log.loc[mask_valid_features, 'Anomaly_Score'] = anomaly_scores

# Step 7: Summarize classified compounds per run
summary_counts = df_valid_ratio.groupby('Classification').size()
# Calculate per-run summary as requested
summary_per_run = df_valid_ratio.groupby(['run_no', 'Classification']).size().
    unstack(fill_value=0)

# --- Output Generation ---

# Text Output: Append to 'compound_classification.md'
output_md_path = '/workspace/compound_classification.md'

with open(output_md_path, 'a') as f:
    f.write("\n---AnalysisStep3:EmpiricalRatioThresholdingandMultivariate
        OutlierDetection---\n")
    f.write(f"RatioThresholds:{ratio_thresholds}\n")
    f.write(f"Counts-Protein:{summary_counts.get('Protein',0)},NucleicAcid:
        {summary_counts.get('NucleicAcid',0)},Mixed:{summary_counts.get('Mixed
        ',0)}\n")

    # Top 10 outliers by anomaly score
    # Filter for actual outliers (score -1 in IsolationForest usually, but we use
    # anomaly score magnitude)
    # The plan asks for top 10 outliers. We sort by Anomaly_Score descending.
    top_outliers = df_valid_log.nlargest(10, 'Anomaly_Score')[['run_no', '
        volume_ml', 'Anomaly_Score']]
    f.write("Top10Outliers(run_no,volume_ml,Anomaly_Score):\n")
    # Replace loop with to_string for efficiency
    f.write(top_outliers.to_string(index=False))
    f.write("\n")

    # Append per-run summary
    f.write("Per-RunClassificationCounts:\n")
    f.write(summary_per_run.to_string())
    f.write("\n")

# Visual Output: 'ratio_classification_scatter.png'
# X: Conc_B_%, Y: Ratio_280_260
# Color by Classification, Mark outliers with 'X'
# Use hexbin or sampling if > 10,000 points

```

```

# Simplify Plotting Data Selection: Initialize plot_df directly from df_valid_log
plot_df = df_valid_log.dropna(subset=['Conc_B_%', 'Ratio_280_260', 'Outlier_Score'
])

# Determine rendering method based on dataset size
n_points = len(plot_df)

plt.figure(figsize=(10, 6))

if n_points > 10000:
    # Random sampling for large datasets to preserve color mapping
    plot_df = plot_df.sample(n=10000, random_state=42)

# Define valid scores mask
valid_scores = plot_df['Outlier_Score'].notna()
non_outliers = plot_df[valid_scores & (plot_df['Outlier_Score'] != -1)]
outliers = plot_df[valid_scores & (plot_df['Outlier_Score'] == -1)]

# Replace loop with single scatter call using color mapping
color_map = {'Protein': 'red', 'Nucleic_Acid': 'blue', 'Mixed': 'green'}
colors = non_outliers['Classification'].map(color_map)

plt.scatter(non_outliers['Conc_B_%'], non_outliers['Ratio_280_260'],
            c=colors, label='Non-Outliers', alpha=0.4, s=10, edgecolors='none')

# Plot outliers
plt.scatter(outliers['Conc_B_%'], outliers['Ratio_280_260'],
            c='black', marker='X', s=100, label='Outlier', zorder=5)

# Add threshold lines
plt.axhline(y=ratio_1st, color='gray', linestyle='--', linewidth=1, label=f'1st_
Percentile_{ratio_1st:.2f}')
plt.axhline(y=ratio_99th, color='gray', linestyle='--', linewidth=1, label=f'99th_
Percentile_{ratio_99th:.2f}')

plt.xlabel('Conc_B_%')
plt.ylabel('Ratio_280_260')
plt.title('Ratio_Classification_and_Outlier_Detection')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('/workspace/ratio_classification_scatter.png', dpi=150)
plt.close()

print("Step_3_completed_successfully.")
###

```

Listing 3: Code_3